

Supplementary Materials: Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking for Shared Execution Services under Deadline-Driven Bursty Load

[First Author Name]¹ and [Second Author Name]^{2,*}

This file holds four kinds of supporting material for the manuscript, in the order the manuscript needs them: the visibility audit of the feature pipeline and the predictor scores on the four non-judging traces (Section S1); the field audit of the main judging trace and two checks on the continuous-integration refit (Section S2); five tables of experimental detail, the selected budget points, the two budget shapes the main guard table no longer prints, the identity residual on all nine cells and the single-server configuration (Section S3); and two deferred proofs, of the workload comparison and of the consistency proposition (Section S4). Section numbers here are S1–S4 and every table is numbered S. A cross-reference without an S — “Section 8.2”, “Table 8”, “Theorem 1” — points into the main manuscript, since a \ref across the two files does not resolve. The traces are the ones the manuscript’s Table 3 names: the CodeBench judging logs [1], the Azure Functions invocation trace [2], the Intel Netbatch logs [3] and the two Firefox continuous-integration pools collected from Mozilla’s public APIs [4,5]. Every figure quoted here is a development figure in the sense of Section 8.2 of the manuscript: nothing in this file comes from a sealed part of any trace, and nothing here was recomputed for it.

S1. Cost Prediction

S1.1. The Visibility Audit in Full

Section 4.2 of the manuscript states that every historical feature is recomputed by streaming the log in availability order and reading each entity’s state as of the query instant, and that 0 rows disagree in any of the eight configurations. This is what that test is.

On 3,000 randomly drawn development jobs all 34 history and relational features are recomputed from scratch under each configuration’s own visibility rule and compared against the pipeline’s values. The same test applied to the natural but wrong construction, in which a feature may use any record with a smaller index, fails on 97.67% of rows under the primary rule, so the test does discriminate. The graph pipeline is checked separately: an index assertion over 863,149 rows finds 0 timeline entries pointing at or after the read, brute-force recomputation on 600 rows matches on all 2,400 root states and all 2,400 neighbour sets, and perturbing or deleting every record after a cut time leaves all earlier predictions unchanged while a positive control moves the later ones.

S1.2. Predictors on the Non-Judging Traces

Table S1 is the cross-trace counterpart of Table 1 of the manuscript. The three figures Section 4.4 quotes from it are the serverless STATIC Spearman correlation and the two CI pools’ ENT AUROC; the AUROC column of Table 11 of the manuscript carries the figure the scheduling results there are read against, which is the ALL row of each trace.

Table S1. The same protocol on the non-judging traces, by feature group. ENT uses the entity's own visible history, STATIC uses only what is declared before the job runs, ALL is both. RMSE and Spearman are on $\log(1 + C)$.

Trace	Features	AUROC	AUPRC	RMSE	Spearman
Serverless invocations	ENT	0.9828	0.816	0.3401	0.953
	STATIC	0.6238	0.084	0.8088	0.021
	ALL	0.9847	0.830	0.3185	0.932
CI pool, hardware A	ENT	0.6334	0.157	0.3964	0.833
	STATIC	0.4990	0.045	0.7659	0.277
	ALL	0.6512	0.177	0.3956	0.837
CI pool, hardware B	ENT	0.9968	0.984	0.0887	0.985
	STATIC	0.6915	0.077	0.6411	0.500
	ALL	0.9970	0.985	0.0860	0.988
Compute farm	ENT	0.8903	0.265	1.4214	0.883
	STATIC	0.8704	0.242	2.2559	0.750
	ALL	0.8902	0.260	1.3485	0.892

S2. Traces and Workload Characterisation

S2.1. The Field Audit of the Main Judging Trace

Section 7.2 of the manuscript states four readings of the main trace. This is the evidence for them, and the two archive defects that follow from reading the trace this way.

The timestamp is a result stamp.

Among later-period runs the gap to the same user's previous event grows one for one with the cost (slope 0.99 of the median gap on C) while the gap to the next event barely moves (slope 0.16), and the feedback after a user kills a run arrives within about 0.1 s whatever its length. A result stamp predicts both readings; a receipt stamp neither.

The whole-second leak.

Stamps are recorded to the whole second while costs are recorded to the microsecond, so the fractional part of a job's cost appears in the gap between its own stamp and the next event, and a feature like "seconds since the user's previous record" reads the label off the clock. We close it by replacing t with $t + u$, where $u \in [0, 1)$ is drawn deterministically from a fixed key and the record's identifier.

The cost field and the limit.

The cost field is wall-clock occupancy of one evaluation including interpreter start-up, read from the 10th percentile of correct submissions (0.12–0.18 s across the two periods) and from its weak dependence on the number of test cases (median $0.196 + 0.007n$ s in the later period). The platform limit is 60 s: the last run to reach it ends at 2019-08-28 10:32, after which the largest observed cost is 59.97 s, with 13 runs in $[59, 60)$ s and none above. Before that date there was no fixed limit, the longest run is 8,901 s, and the 1,189 runs that end with a kill are almost all above 360.8 s, pointing at a container watchdog rather than judging demand.

Co-ending groups, zero-cost blocks and two short archives.

Groups of runs by the same user that end in the same second are 7.53% of blocks and 26.28% of capped work; every prediction table in the manuscript is repeated on rows outside such groups. A further 1,671 blocks record a cost of exactly zero with empty outputs; they leave the simulated trace and stay in the features, the labels and every total. The 2019-2 archive stops at 2019-09-19, months before that term's assessments end, and 2020-1 is short by about 3% of its blocks; both serve per-submission modelling and are excluded from anything that treats a term as a whole.

Against the published release statistics.

We match the class count in 18 of 18 terms and the student and exercise counts in 16 of 18, and parse 4.08% fewer execution blocks than the official column sums, whose printed totals disagree with those same columns by -248 exercises and $+246,246$ blocks. We quote the columns.

S2.2. *The Continuous-Integration Refit: Sharpness and Circularity*

Section 7.3 of the manuscript reports the held-out refit of the effective capacity k_{eff} and the setup term on the two Firefox CI pools. Two checks on that fit sit here.

The objective is sharp in k rather than flat: on a step-of-one grid only 5 to 8 values of k put the simulated mean wait within 10% of the recorded one, so the capacity the fit chooses is not one of many equally good answers.

The circularity of the original whole-window fit can be read off the two pools directly. On pool A the whole-window and held-out rows are almost identical, because every split picks nearly the same capacity; on pool B the reverse split’s held-out capacity gives a mean ratio of 1.14 on its test part where the whole-window capacity gives 0.99. The difference between those two numbers is the size of the circularity that fitting on the reported window would have hidden.

S3. Experiments

S3.1. *What the Selection Rule Chose*

Table 5 of the manuscript gives the three pre-stated grids, the feasibility rule, the objective and the tie-break, and names the shape the rule selected at each promise. Table S2 gives the selected points in full and the nine per-shape winners. The same values are in `protocol_lock.draft.json`, which is the artefact the selection was frozen into.

Table S2. The three selected budget points and the nine per-shape grid winners. Budgets in work scale as $k/4$; the harm limit is $G/2$, the feasibility threshold of the selection rule. “Feasible” counts the grid points that met it in all 15 validation cells.

Item	Value
Selected, $G = 300$ s	age-relative: $B_0 = 60k/4$ s, $\eta = 0.50$, $\gamma = 0$; worst-cell gap 0.4251, worst-cell harm 122.8 s against a 150 s limit, 44 of 99 candidate points feasible
Selected, $G = 600$ s	age-relative: $B_0 = 120k/4$ s, $\eta = 0.75$, $\gamma = 0$; worst-cell gap 0.7216, worst-cell harm 293.3 s against 300 s, 70 of 107 feasible
Selected, $G = 1200$ s	queue-length: $B_0 = 0$, $\eta = 0$, $\gamma = 16k/4$ s; worst-cell gap 0.7283, worst-cell harm 145.3 s against 600 s, 95 of 111 feasible
Per-shape winners	constant: $B_0 = 329.671, 981.126, 2,029.958 k/4$ s at the three promises; age-relative: $(60, 0.50), (120, 0.75), (360, 0.75)$; queue-length: $\gamma = 16k/4$ s with $B_0 = 0, 120k/4$ s, 0

S3.2. *The Two Budget Shapes the Main Table No Longer Prints*

Section 8.4 of the manuscript compares the three budget shapes at a fixed promise through the paired differences, which is how the comparison is read. Table S3 gives the age-relative and queue-length rows themselves, in the format of Table 8 and in the same cells; the Guard(G), Guard-fixed(G), Fixed(G) and Skip(G) rows stay there. One of the three shape winners coincides with Guard(G) at each promise, which is why some rows below repeat a row of Table 8.

Table S3. The age-relative and queue-length winners at three promises and three loads, in the format of Table 8 of the manuscript. Excess and harm are the worst over the five overlays, the rest are means; budgets scale as $k/4$.

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$								
	Guard-age(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-queue(300)	300	15.27	0.723 [0.697, 0.742]	44.8	193.4	94.3	21.60
	Guard-age(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-queue(600)	600	14.88	0.744 [0.718, 0.765]	46.1	504.5	106.0	0.16
	Guard-age(1200)	1200	14.73	0.751 [0.723, 0.773]	46.5	1,049.1	237.0	0.01
	Guard-queue(1200)	1200	14.84	0.746 [0.719, 0.768]	46.2	907.8	94.3	22.01
$\rho = 0.8, k = 5$								
	Guard-age(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-queue(300)	300	54.69	0.753 [0.713, 0.777]	54.0	210.7	117.6	17.53
	Guard-age(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-queue(600)	600	47.69	0.834 [0.814, 0.847]	59.9	508.5	117.7	1.35
	Guard-age(1200)	1200	45.16	0.864 [0.844, 0.876]	62.0	1,102.9	370.5	0.29
	Guard-queue(1200)	1200	46.85	0.844 [0.823, 0.857]	60.6	1,104.9	117.6	9.93
$\rho = 1.0, k = 4$								
	Guard-age(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-queue(300)	300	169.22	0.453 [0.368, 0.607]	38.1	231.4	144.8	30.99
	Guard-age(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-queue(600)	600	91.09	0.793 [0.758, 0.833]	66.6	515.8	123.4	7.08
	Guard-age(1200)	1200	72.52	0.874 [0.851, 0.893]	73.4	1,117.3	453.3	0.81
	Guard-queue(1200)	1200	82.67	0.830 [0.801, 0.857]	69.7	1,110.4	144.8	6.21

S3.3. The Residual on All Nine Cells

Table 10 of the manuscript prints the three Guard(600) cells and one reversed score. Table S4 gives all nine. The reading Section 8.5 takes from them is that the residual is a property of the schedule and barely moves with the policy: inside each load the three policies agree to the third decimal of the ratio.

Table S4. The residual $D_i = k(W_P[i] - W_{FCFS}[i]) - (\text{In}_i - \text{Out}_i)$ of Theorem 1 of the manuscript on overlay 0, 17,634,760 jobs per cell. Ratio is $\max |D_i|$ over the bound $2(k-1)L$, which is $14L$, $8L$ and $6L$ at the three loads; R^2 and the worst error are for $(\text{In}_i - \text{Out}_i)/k$ against the realised excess over first-come first-served.

Load	Policy	$\max D_i /L$	Ratio	$D_i = 0$	R^2	worst err. (s)
$\rho = 0.5, k = 8$	SPJF-E	6.788	0.485	95.6%	0.985	50.9
	Guard(600)	6.788	0.485	95.6%	0.983	50.9
	Reversed score	6.788	0.485	95.6%	0.996	50.9
$\rho = 0.8, k = 5$	SPJF-E	3.922	0.490	87.2%	0.996	47.1
	Guard(600)	3.922	0.490	87.2%	0.995	47.1
	Reversed score	3.881	0.485	87.5%	0.999	46.6
$\rho = 1.0, k = 4$	SPJF-E	3.082	0.514	81.1%	0.998	46.2
	Guard(600)	3.082	0.514	81.1%	0.998	46.2
	Reversed score	2.994	0.499	81.7%	1.000	44.9

S3.4. A Single Server

Section 8.6 of the manuscript reports the single-server configuration through five gap figures, three harm ratios and two realised excesses. Table S5 is the configuration in full. The parameters are the ones the selection rule chose on the 15 multi-server validation cells; $k = 1$ is not among them, and we do not re-tune on it.

The firing rate of the three ungarded policies is printed ---: the column counts dispatch epochs at which the guard overrides the base policy, and these three runs have no

guard. Scoring first-come first-served at 100% by the convention that every first-come first-served dispatch serves the queue head, as an earlier version of this table did, compares it with the guarded runs on a quantity it does not have.

Table S5. Single server, busy-hour $\rho = 0.80$, 3,209,347 jobs. Waits and excesses in seconds. The parameters are the ones the rule selected on the multi-server validation cells; this configuration is not among them. The skip count is $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$.

Policy	Promise	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
FCFS	—	221.27	0.000	0.0	0.0	—
SJF on true sizes	—	45.32	1.000	2,054.5	777.9	—
SPJF-E	—	60.91	0.911 [0.854, 0.933]	2,520.3	656.4	—
Guard(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-fixed(300)	300	125.68	0.543 [0.207, 0.718]	140.3	119.5	17.09
Guard-age(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-queue(300)	300	105.15	0.660 [0.532, 0.715]	297.9	53.9	29.15
Fixed(300)	300	83.18	0.785 [0.584, 0.865]	297.9	246.3	6.33
Skip(300), $\kappa = 5$	300	220.39	0.005 [-0.002, 0.019]	95.1	81.8	73.17
Guard(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-fixed(600)	600	82.10	0.791 [0.602, 0.868]	303.0	246.3	6.16
Guard-age(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-queue(600)	600	91.52	0.737 [0.640, 0.812]	582.0	81.8	3.43
Fixed(600)	600	68.48	0.868 [0.783, 0.906]	593.4	563.6	2.29
Skip(600), $\kappa = 10$	600	219.89	0.008 [-0.004, 0.037]	100.9	83.2	66.98
Guard(1200)	1200	98.00	0.701 [0.640, 0.742]	1,157.4	53.9	26.15
Guard-fixed(1200)	1200	69.03	0.865 [0.789, 0.905]	564.2	509.3	2.57
Guard-age(1200)	1200	63.80	0.895 [0.835, 0.920]	1,182.3	366.4	0.68
Guard-queue(1200)	1200	98.00	0.701 [0.640, 0.742]	1,157.4	53.9	26.15
Fixed(1200)	1200	63.19	0.898 [0.837, 0.923]	1,180.4	656.4	0.45
Skip(1200), $\kappa = 20$	1200	217.64	0.021 [-0.008, 0.084]	168.0	97.7	58.67

S4. Deferred Proofs

Appendices A.1 and A.5 of the manuscript give the idea of each argument below and point here for the argument itself. Both are continuity-and-left-limit arguments a reader will not return to, which is why they sit outside the manuscript; the constructions the contribution rests on stay there, in Appendices A.2–A.4, A.6 and A.7. Nothing here was restated or re-derived for this file: the text is the text the manuscript carried until the seventh revision pass. The notation is the manuscript's, $U_Q(t)$ being the unfinished work at time t under policy Q and $N_Q(t)$ the number of jobs present.

S4.1. The Workload Comparison

Proof of Lemma 1 of the manuscript. Put $D = U_{Q_1} - U_{Q_2}$. Arrivals add the same jump to both terms, so D is continuous, and between arrivals it changes at rate $-(\min(N_{Q_1}, k) - \min(N_{Q_2}, k))$; it is therefore piecewise linear with breakpoints among the finitely many arrival and completion instants of the two runs.

If $U_{Q_1}(u) > (k - 1)L$ then more than $k - 1$ jobs are present under Q_1 , each carrying at most L , so every server of Q_1 is busy and $\min(N_{Q_1}, k) = k \geq \min(N_{Q_2}, k)$: D does not increase at u . Fix t and let $S = \{u \leq t : U_{Q_1}(u) \leq (k - 1)L\}$, which is non-empty because U_{Q_1} vanishes before the first arrival, and put $t_0 = \sup S$. The supremum need not be attained, since U_{Q_1} jumps upwards at arrivals, so we use the left limit: there are $u_n \uparrow t_0$ with $U_{Q_1}(u_n) \leq (k - 1)L$, hence $U_{Q_1}(t_0^-) \leq (k - 1)L$ and, D being continuous, $D(t_0^-) \leq U_{Q_1}(t_0^-) \leq (k - 1)L$ because $U_{Q_2} \geq 0$. On $(t_0, t]$ we have $U_{Q_1} > (k - 1)L$, so D does not increase there, and $D(t) \leq (k - 1)L$. Exchanging Q_1 and Q_2 gives the other direction. That the constant cannot be lowered is the cascade of Appendix A.2 of the manuscript, which reaches $(k - 1)L(1 - ((k - 1)/k)^m)$ after m rounds. At $k = 1$ the bound

reads $|D| \leq 0$ and is attained everywhere: one server executes work at rate 1 whenever any job is present, so $U_{Q_1} \equiv U_{Q_2}$. $\square$

Proof of the strict inequality in Lemma 1, for $k \geq 2$. Keep the notation above, so $D = U_{Q_1} - U_{Q_2}$ is continuous and piecewise linear with finitely many pieces. Suppose $D(t) = (k-1)L$ for some t and let t_1 be the smallest such t ; it exists because $\{D = (k-1)L\}$ is closed and $D(0) = 0 < (k-1)L$, which uses $L > 0$ and $k \geq 2$.

Let I be the linear piece immediately to the left of t_1 , which has positive length. Its slope is strictly positive: a negative slope would put $D > (k-1)L$ somewhere on I and a zero slope would put $D = (k-1)L$ on all of I , and either contradicts the minimality of t_1 . The slope of D is $-(\min(N_{Q_1}, k) - \min(N_{Q_2}, k))$, so $\min(N_{Q_1}, k) < \min(N_{Q_2}, k)$ on I and therefore $N_{Q_1} \leq k-1$ there.

With at most $k-1$ jobs present and Q_1 work-conserving, none of Q_1 's jobs waits, so all of them are in service; hence U_{Q_1} falls at rate N_{Q_1} on I , and $U_{Q_1} \leq (k-1)L$ on I because each of the at most $k-1$ present jobs carries at most L . Taking left limits at t_1 and using the continuity of D ,

$$U_{Q_1}(t_1^-) - U_{Q_2}(t_1^-) = (k-1)L, \quad U_{Q_1}(t_1^-) \leq (k-1)L, \quad U_{Q_2} \geq 0,$$

which forces $U_{Q_2}(t_1^-) = 0$ and $U_{Q_1}(t_1^-) = (k-1)L$. Since $L > 0$ and $k \geq 2$, this gives $N_{Q_1} \geq 1$ on I — with $N_{Q_1} = 0$ we would have $U_{Q_1}(t_1^-) = 0 = (k-1)L$ — so U_{Q_1} is strictly decreasing on I and $U_{Q_1}(t_1^-) < \sup_I U_{Q_1} \leq (k-1)L$, a contradiction. Exchanging Q_1 and Q_2 gives the other direction. Both steps that invoke $(k-1)L > 0$ fail at $k = 1$, where the statement is false: there $U_{Q_1} \equiv U_{Q_2}$ and the bound 0 is attained at every instant. $\square$

S4.2. Consistency, and Why the Converse Fails

Proof of Proposition 1 of the manuscript. Induct on dispatch epochs: up to the first epoch at which the two runs could differ their completion histories agree, hence so does $\text{over}[\cdot]$, so $E(t)$ is empty in the wrapped run and it copies A . The first consequence uses $\text{over}[q](t) \leq \text{In}_q^A$, a completed overtaker of q having been dispatched while q waited; the second uses Corollary 2 of the manuscript. $\square$

The converse fails, so this is not an equivalence. Take $k = 1$, two jobs of service 1 at $t = 0$, the base policy “higher rank first”, and $B = 1$. Both runs dispatch in the order $(1, 0)$, yet at the second dispatch the waiting job has $\text{over} = 1 \geq B$, so the guard fires and happens to choose what the base would have chosen. Nor is there multiplicative consistency: with $k = 1$, one job of service L followed by m jobs of service 1 all at $t = 0$, the wrapper with $B = 0$ is first-come first-served while the base ordering by service time is not, and the ratio of mean waits is $(2L + m - 1)/(m + 1)$, unbounded in L .

References

1. Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19.
2. Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021.
3. Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/1_intel_netbatch/, 2012.
4. Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19.
5. Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19.